

Queue-Aware Market Making on a Real NASDAQ Order-Book Sample

A reproducible pipeline-validation study

Charles Backman - Quant Systems Lab

This paper validates a real-data research pipeline; it does not claim persistent profitability. The public sample contains one AAPL session and truncated Level-5 state.

Abstract

We evaluate fixed-spread, Avellaneda-Stoikov, queue-aware, toxicity-aware, and latency-aware market-making policies using a single shared replay engine and a strictly chronological train, validation, embargo, and test design. The pipeline ingests 301,587 synchronized LOBSTER messages and book states, converts exchange-local time to UTC nanoseconds, records immutable data and configuration fingerprints, reconciles reconstructed state, models queue depletion and latency, forces terminal liquidation, and independently recomputes cash from fills. All policies lose money on the held-out sample. This negative result is retained and interpreted as evidence about execution sensitivity, not as a failed software demonstration.

1. Research question and hypotheses

The primary research question is whether explicit queue state improves executable outcomes after fees, latency, inventory risk, and adverse selection. The central comparison is queue-aware quoting against fixed-spread and Avellaneda-Stoikov baselines. Toxicity- and latency-aware policies test whether contemporaneous signed flow and stale-state risk reduce adverse fills.

2. Data and provenance

Provider	LOBSTER public demonstration sample
Instrument	AAPL, NASDAQ, 2012-06-21
Rows	301,587 canonical messages / 301,587 snapshots
Dataset fingerprint	a985e2d3cb1e14abf10f9b8a60177d7fdcf7244574a3cc137c312e00cca1ecce
Config fingerprint	59baf316d544cc64b312969f33222e76fb5af450ac5e9310b73c3f87fdd2ca98
Reconstruction match	89.17%

The repository never redistributes raw provider files. File hashes and provenance are recorded in a local manifest. Level-5 truncation means a price can enter the visible range without its original resting history. The reconstructor counts such mismatches and reseeds from synchronized provider state.

3. Chronological design

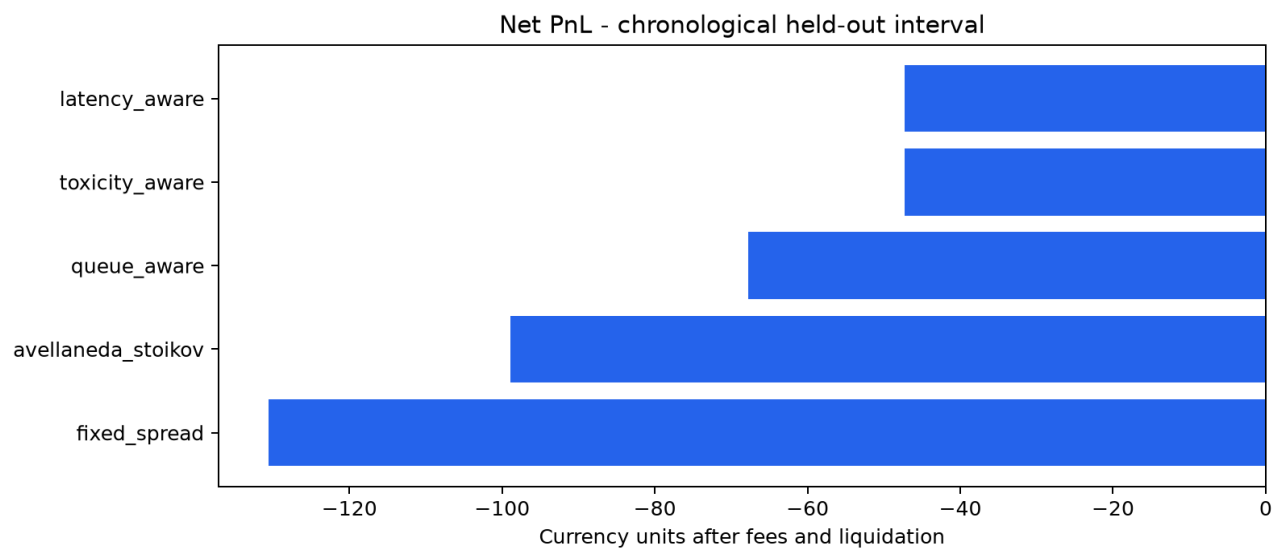
The session is divided into training, validation, and untouched test intervals with one-second embargoes. Training estimates descriptive intensities, spread transitions, volatility, and short-horizon signed price response. Validation selects queue-ahead fraction. The test interval is evaluated with the frozen choice. Validation selected a queue-ahead fraction of 1.00.

queue_ahead_fraction	net_pnl	risk_score
0.000000	12.949074	0.219476
0.500000	-31.740610	-0.577102
1.000000	15.671772	0.474902

4. Shared execution model

Every policy receives identical historical messages, inventory limits, order size, quote cadence, fees, queue assumptions, and liquidation rules. Historical cancels reduce queue-ahead depth; visible executions consume remaining queue before filling the counterfactual agent. Observations are delayed by the configured latency. The model assumes a small agent whose orders do not alter subsequent market events.

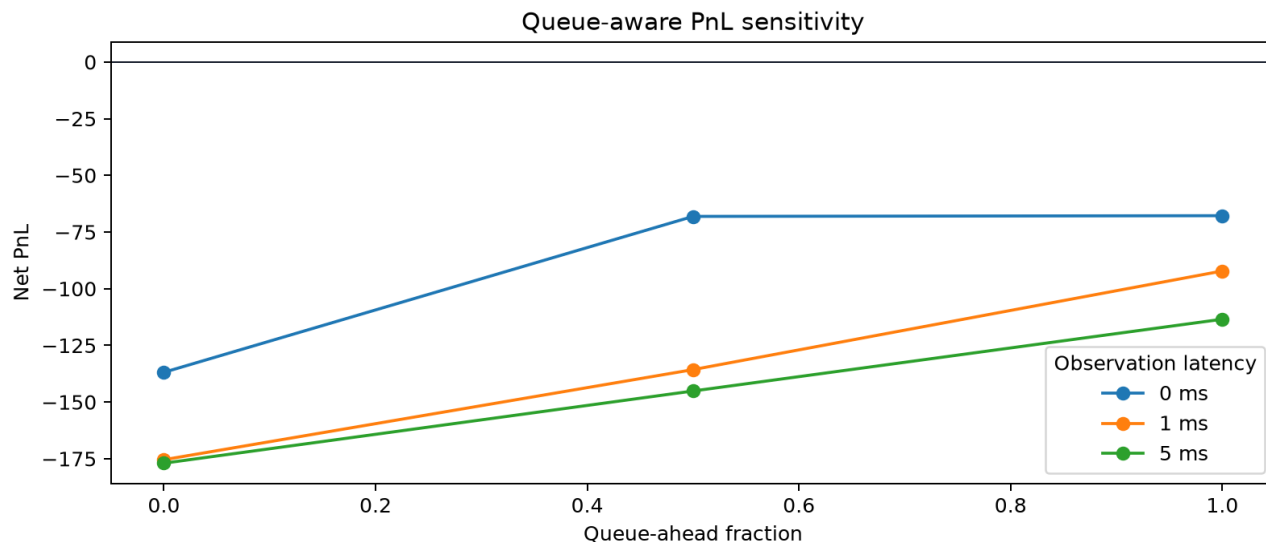
5. Held-out results



strategy	net_pnl	fill_rate	adverse_selection_cost	max_abs_inventory	max_drawdown
fixed_spread	-130.629500	0.005384	0.011043	99.000000	142.232520
avellaneda_stoikov	-98.875712	0.004596	0.006305	91.000000	117.216487
queue_aware	-67.801848	0.004070	0.017819	76.000000	90.805785
toxicity_aware	-47.337183	0.004273	0.009515	52.000000	71.007982
latency_aware	-47.337183	0.004273	0.009515	52.000000	71.007982

All five policies have negative net PnL. The toxicity-aware and latency-aware specifications lose the least in the base scenario, while queue-aware quoting improves on fixed spread but remains unprofitable. These rankings are descriptive for one held-out intraday interval and are not evidence of persistent alpha.

6. Sensitivity and failure analysis



Queue position, latency, and fee multipliers are varied over the frozen grid. The policy comparison is therefore not conditional on a single favorable queue assumption. Failure cases include inventory build-up, adverse post-fill price movement, and profitability that is dominated by fee rebates or terminal liquidation.

7. Accounting and governance

The largest absolute cash-ledger reconciliation error is $3.638e-11$. Each publishable run records the data fingerprint, frozen configuration fingerprint, Git commit, dirty-tree status, seed, command, environment, and artifact hashes in an append-only experiment directory.

8. Limitations

The public sample covers one 2012 AAPL session, only five visible levels, no distinct receive timestamp, and an illustrative fee tier. Original queue priority outside the visible range and hidden liquidity are not fully observed. Intraday block bootstrap intervals cannot replace session-level inference. A final paper requires licensed multi-session L2 or L3 data, true receive timestamps, provider-specific reconciliation, and a later untouched test interval.

9. Conclusion

The contribution of this study is an auditable empirical research system rather than a profitable result. It demonstrates that real event data can be ingested, reconstructed, split chronologically, replayed through five comparable policies, stress-tested, and reconciled without hiding negative outcomes. The system is ready to receive a larger licensed dataset.

References

Avellaneda, M. and Stoikov, S. (2008). High-frequency trading in a limit order book. Quantitative Finance.
Huang, W. and Polak, T. LOBSTER output and public NASDAQ sample documentation.
Cartea, A., Jaimungal, S., and Penalva, J. (2015). Algorithmic and High-Frequency Trading.